# Sentiment of Movies


## Lin Guo, Thomas Stranick

Data Wrangling and Husbandry, Stevenson Bolivar Atuesta


5/7/25

# Executive Summary


If you were making a movie, releasing a product, or just putting something out there for the world to see, wouldn't you want to know it would be a failure beforehand? Or at least know how to change your product so that it will become a successful release?

Utilizing audience sentiment from movie trailers to help predict how much money your movie will earn, and in turn how successful it will be, could be a beneficial strategy for answering these questions. YouTube is the primary site for companies to release trailers and for audience to give feedback in the way of comments. So we primarily utilized the YouTube API and Box Office Mojo to find audience sentiment of trailers on YouTube and compared that with the gross earnings of a movie's release. Using these resources, we would like to answer the question; Can audience sentiment on a pre-release movie trailer from YouTube, predict how much revenue or how successful a movie will be post-release?

In our analysis, we combined and cleaned data from 2023 and 2024 movies, gathering YouTube links to query the API, removing stop-words and non-words for sentiment analysis, and readying box office data to be interpretable in our model. Our key takeaways from this analysis, mainly showed that using solely audience sentiment to predict gross revenue was not feasible. Many high grossing movies did not have the best audience sentiment scores, and some that had higher scores were lower grossing. Although it may not be possible to predict how well a movie will do based soley on audience sentiment from YouTube trailers, it could be a useful covariate in further analysis of the same issue.


# Introduction

Every year movies are released that are considered "box office bombs", most recently being Furiosa: A Mad Max Saga (2024) and Joker: Folie a Deux (2024). These movies lost $120 and $144 million respectively, when comparing their gross revenue to the cost to produce the film [1]. In anticipation of these movies, studios release trailers to show a preview of their movie that is coming out soon. Audiences globally get a chance to see a snippet of the movie before it releases and give their impressions and opinions. We can use sentiment analysis of these opinions to see how they think and feel about the movie. How can we predict the overall gross revenue of a movie before it is released by using sentiment analysis from pre-release trailers?

Using the audiences reaction to trailers could save studios millions of dollars, by making the movie more likable by the audience, or not releasing it all together. One famous example of using audience feedback to benefit a movie was the first Sonic the Hedgehog movie, where audience backlash over character design forced the creators to alter it [2]. Now the franchise is worth over 1 billion dollars and releasing it's third movie in the series [3]. Sentiment analysis of audience reaction could also generally translate to any business use case. If a company is releasing a product, it would be beneficial to know what the general audience thinks of it before selling. This way, the business can make its product more appealing to the customer, thus increasing sales and profitability.

For our sentiment analysis we chose to utilize YouTube comments left on trailers for the top grossing films of 2023 and 2024 that we can extract using the Youtube API [4]. These comments contain positive or negative opinions about the trailer that we can then analyze for our sentiment analysis and use in a prediction model. The links for each Youtube video were gathered manually, as there was no dataset readily available [5]. For our box office gross revenue data, we used web scraping on the Box Office Mojo website and found the gross revenue data for 2023 and 2024 movies [6,7].

Utilizing the comment and gross revenue datasets from 2023 and 2024, we can combine them and create a model that can predict gross revenue from emotional sentiment of trailers. Here, we will use 2023 to train a model, and 2024 to test and compare the results. This model as well as in depth sentiment analysis, will be the main objectives for our analysis, and can help movie studios determine if their movie will be a success or a flop. We will also take a closer look at some of the most extreme positive and negative sentiment trailers, and see how much those movies made in gross revenue.

# Data Wrangling & Cleaning

Cleaning the gross revenue data from Box Office Mojo was relatively straightforward. To start, we removed movies that were re-releases, since audiences would already know what they would be seeing. After that, we needed to convert some of the information into usable data, such as removing some of the characters from the gross revenue column to make it a number, and making sure that the dates contained their year and were in the correct format. We also only wanted to keep the release date, movie title, and gross revenue data to use in our analysis.

Below and throughout this section we show the comparisons for 2024 data, because the data cleaning for 2023 and 2024 is visually the same.

**Box Office Mojo Before Tidying:**

```
## # A tibble: 200 × 11
##     Rank Release        Genre Budget `Running Time` Gross Theaters `Total Gross`
##    <int> <chr>          <chr> <chr>  <chr>          <chr> <chr>    <chr>
##  1     1 Inside Out 2   -     -      -              $652… 4,440    $652,980,194
##  2     2 Deadpool & Wo… -     -      -              $636… 4,330    $636,745,858
##  3     3 Wicked         -     -      -              $432… 3,888    $473,231,120
##  4     4 Moana 2        -     -      -              $404… 4,200    $460,405,297
##  5     5 Despicable Me… -     -      -              $361… 4,449    $361,004,205
##  6     6 Beetlejuice B… -     -      -              $294… 4,575    $294,100,435
##  7     7 Dune: Part Two -     -      -              $282… 4,074    $282,144,358
##  8     8 Twisters       -     -      -              $267… 4,170    $267,762,265
##  9     9 Godzilla x Ko… -     -      -              $196… 3,948    $196,350,016
## 10    10 Kung Fu Panda… -     -      -              $193… 4,067    $193,590,620
## # i 190 more rows
## # i 3 more variables: `Release Date` <chr>, Distributor <chr>, Estimated <chr>
```

**Box Office Mojo After Tidying:**

```
## # A tibble: 200 × 3
##    movie                              gross release_date
##    <chr>                              <dbl> <date>
##  1 Inside Out 2                     652980194 2024-06-14
##  2 Deadpool & Wolverine             636745858 2024-07-26
##  3 Wicked                           432943285 2024-11-22
##  4 Moana 2                          404017489 2024-11-27
##  5 Despicable Me 4                  361004205 2024-07-03
##  6 Beetlejuice Beetlejuice          294100435 2024-09-06
##  7 Dune: Part Two                   282144358 2024-03-01
##  8 Twisters                         267762265 2024-07-19
##  9 Godzilla x Kong: The New Empire  196350016 2024-03-29
## 10 Kung Fu Panda 4                  193590620 2024-03-08
## # i 190 more rows
```

The YouTube comment data was a bit trickier. After gathering all of the Youtube video links for each movie on the Box Office Mojo list created, we could then utilize the Youtube API to extract the comments from each trailer. First, we joined the Box Office Mojo dataset on the YouTube video link data, and compared all comments with the movie release provided by the Box Office Mojo dataset. Some of the trailers did not contain comments, as the channel owner can turn them off on the video. We removed these from our analysis.

**YouTube Codes Joined on Box Office Mojo Data:**

```
## # A tibble: 90 × 4
##    movie                              code           gross release_date
##    <chr>                              <chr>          <dbl> <date>
##  1 Inside Out 2                       LEjhY15eCx0 652980194 2024-06-14
##  2 Deadpool & Wolverine               73_1biulkYk 636745858 2024-07-26
##  3 Wicked                             6COmYeLsz4c 432943285 2024-11-22
##  4 Beetlejuice Beetlejuice            CoZqL9N6Rx4 294100435 2024-09-06
##  5 Dune: Part Two                     Way9Dexny3w 282144358 2024-03-01
##  6 Twisters                           wdok0rZdmx4 267762265 2024-07-19
##  7 Godzilla x Kong: The New Empire    lV1OOlGwExM 196350016 2024-03-29
##  8 Bad Boys: Ride or Die              hRFY_Fesa9Q 193573217 2024-06-07
##  9 Kingdom of the Planet of the Apes  XtFI7SNtVpY 171130165 2024-05-10
## 10 Gladiator II                       4rgYUipGJNo 164554921 2024-11-22
## # i 80 more rows
```

We also ran into a few roadblocks. We wanted to make sure we were only looking at comments that were left before the release of the movie, since we did not want sentiments from someone who had already seen it.

**After Getting Comments from YouTube API:**

```
## # A tibble: 90 × 3
##    movie                              gross data
##    <chr>                              <dbl> <list>
##  1 Inside Out 2                     652980194 <df [75 × 2]>
##  2 Deadpool & Wolverine             636745858 <df [68 × 2]>
##  3 Wicked                           432943285 <df [72 × 2]>
##  4 Beetlejuice Beetlejuice          294100435 <df [87 × 2]>
##  5 Dune: Part Two                   282144358 <df [76 × 2]>
##  6 Twisters                         267762265 <df [75 × 2]>
##  7 Godzilla x Kong: The New Empire  196350016 <df [72 × 2]>
##  8 Bad Boys: Ride or Die            193573217 <df [68 × 2]>
##  9 Kingdom of the Planet of the Apes 171130165 <df [74 × 2]>
## 10 Gladiator II                     164554921 <df [77 × 2]>
## # i 80 more rows
```

**Example of Extracted Comments:**

```
##   DatePosted
## 1 2024-03-07
## 2 2024-03-07
## 3 2024-03-07
##
Comment
## 1                                I like the detail that envy is small because it makes you feel small and em
barrassment is big because you feel like you&#39;re super noticable
## 2                                                                        I hope that Brazilian helicopter pilot
from Riley’s mom’s flashback makes an in person cameo in this one.
## 3 I love how Riley&#39;s mom doesn&#39;t have those extra emotions, meaning she grew out of them and she&#39;s
comfortable showing her basic emotions without overthinking stuff.
```

On the remaining movies, we then needed to separate each word in the comment for our sentiment analysis. YouTube comments contain a number of different things such as video time stamps, other links, and emojis, that needed to be removed. We also needed to remove stop words such as “the”, “and”, or “but”, since these would not convey emotion.

**After Separating Comments by Words:**

```
## # A tibble: 90 × 3
##    movie                              gross data
##    <chr>                              <dbl> <list>
##  1 Inside Out 2                     652980194 <df [834 × 1]>
##  2 Deadpool & Wolverine             636745858 <df [491 × 1]>
##  3 Wicked                           432943285 <df [784 × 1]>
##  4 Beetlejuice Beetlejuice          294100435 <df [802 × 1]>
##  5 Dune: Part Two                   282144358 <df [1,070 × 1]>
##  6 Twisters                         267762265 <df [885 × 1]>
##  7 Godzilla x Kong: The New Empire  196350016 <df [815 × 1]>
##  8 Bad Boys: Ride or Die            193573217 <df [501 × 1]>
##  9 Kingdom of the Planet of the Apes 171130165 <df [614 × 1]>
## 10 Gladiator II                     164554921 <df [530 × 1]>
## # i 80 more rows
```

**Example of Comments by Word:**

```
##            word
## 1        detail
## 2          envy
## 3         makes
## 4          feel
## 5  embarrassment
## 6          feel
## 7         super
## 8     noticable
## 9          hope
## 10    brazilian
```

Once we had only meaningful words, these could be analyzed by the “bing” sentiment dataset and determined to be positive or negative. We took the difference between the amount of words identified as postivie, by the amount identified as negative as our sentiment score. This would indicate the overall sentiment of the movie based on the trailer over all comments collected.

**After Calculating Sentiment Scores:**

```
## # A tibble: 90 × 3
##    movie                              gross score
##    <chr>                              <dbl> <int>
##  1 Inside Out 2                     652980194     2
##  2 Deadpool & Wolverine             636745858    27
##  3 Wicked                           432943285    45
##  4 Beetlejuice Beetlejuice          294100435    55
##  5 Dune: Part Two                   282144358    81
##  6 Twisters                         267762265    40
##  7 Godzilla x Kong: The New Empire  196350016    38
##  8 Bad Boys: Ride or Die            193573217   -10
##  9 Kingdom of the Planet of the Apes 171130165    70
## 10 Gladiator II                     164554921     1
## # i 80 more rows
```

Finally, we combined 2023 and 2024 to make one complete dataset. From the Box Office Mojo data, we could keep the movie title, gross revenue, and movie release date information. From the YouTube comment data, we could keep the Youtube link, the created word list, and the overall sentiment values calculated from each word list. In our model, we can use the sentiment analysis value to predict our gross revenue data.

**Combined movies table with years:**

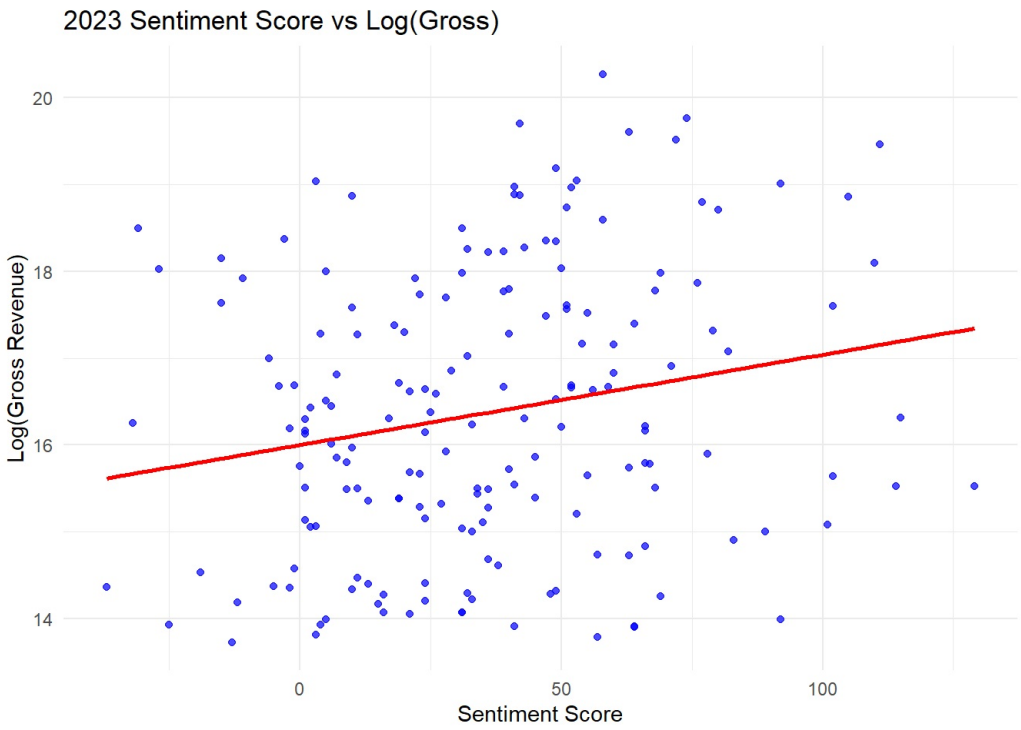
```
## # A tibble: 267 × 4
##    movie                                    gross score  year
##    <chr>                                    <int> <int> <dbl>
##  1 Nefarious                              5433685     1  2023
##  2 Broker                                 1001892     3  2023
##  3 The Wandering Earth II                 3444487     2  2023
##  4 Sound of Freedom                     184177725     3  2023
##  5 Freelance                              5314136     9  2023
##  6 Lost in the Stars                      1721446    -2  2023
##  7 Jules                                  1924922    11  2023
##  8 The Lost King                          1189307     5  2023
##  9 Anatomy of a Fall                      3742039     1  2023
## 10 Creation of the Gods I: Kingdom of Storms   1686246    10  2023
## # i 257 more rows
```

Please refer to Appendix B on Data Cleaning & Transformation for more technical details. And Appendix C for reproducable data wrangling code.

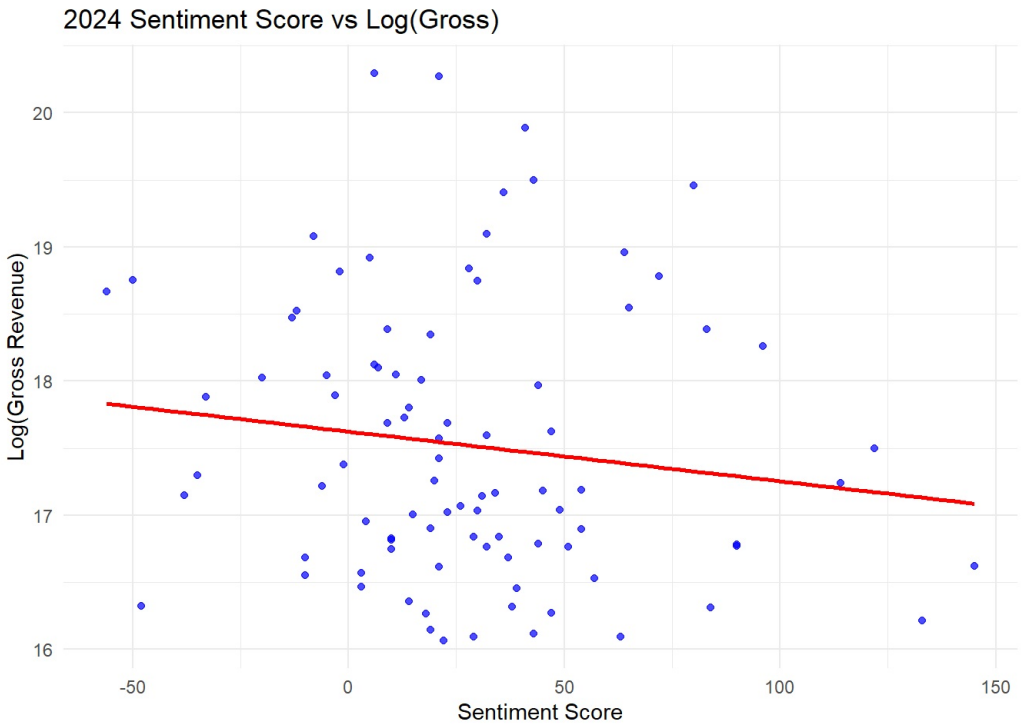
# Exploratory Data Analysis (EDA)

At a first glance, between the two years there showed a different trend in the data of gross by sentiment score. In 2023, there was a slightly increasing trend between the two key variables, in 2024 there was a slight decreasing trend. In the following trend graphs, we have normalized the gross revenue using the log function.

**2023 Trend:**



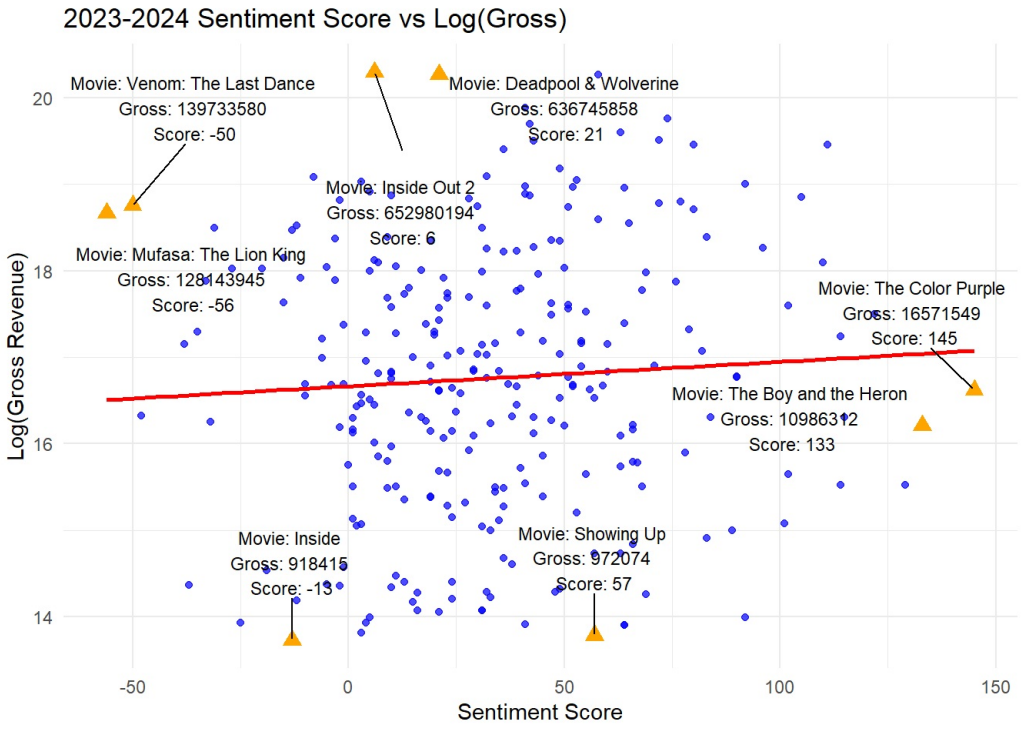

**2024 Trend**




Looking at the combined data below, we can see that the movies are dispersed and the trend line is slightly upward. This indicates that there is not a strong trend in predicting the gross revenue from the sentiment score of the trailers of the movies.

We can look at some of the outliers to see some of the extreme differences in gross and sentiment score. “Inside Out 2” which was the highest grossing movie, only had a sentiment score of 6. The second lowest grossing movie “Showing Up”, had a higher sentiment score of 57. A 51 point positive difference. Similarly, the highest sentiment scored movie “The Color Purple” at +145, had a gross earnings of $16,571,549. Whereas the lowest sentiment scored movie “Mufasa: The Lion King” at -56 had a gross earnings of $128,143,945. A difference of $111,572,396, which is a significant amount of money.

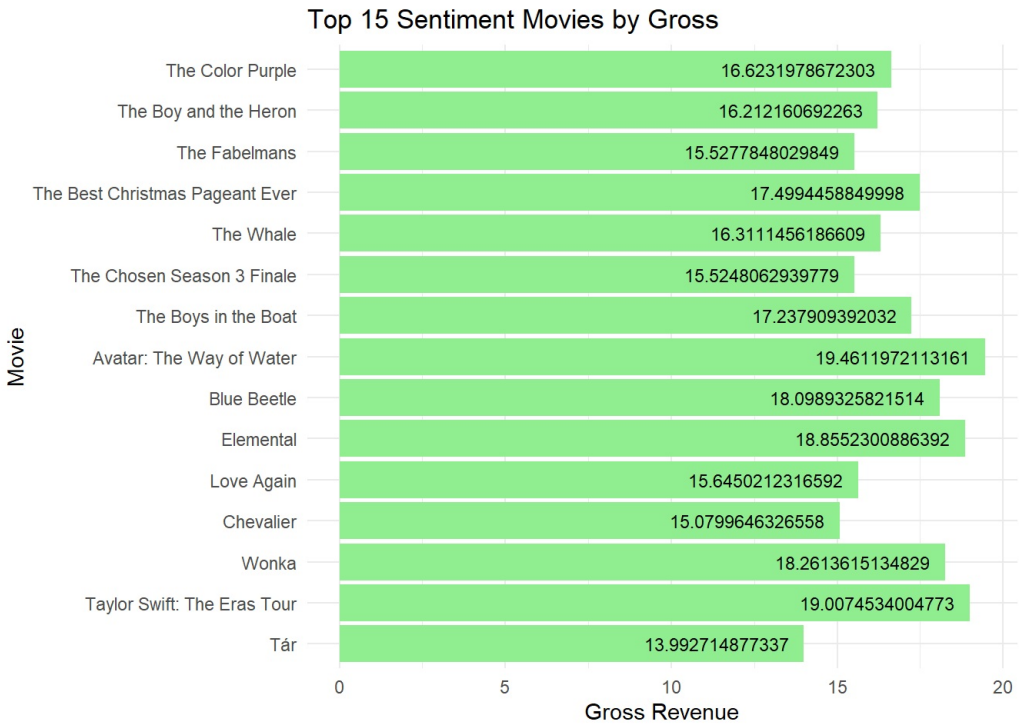
**Overall Trend and Outliers:**



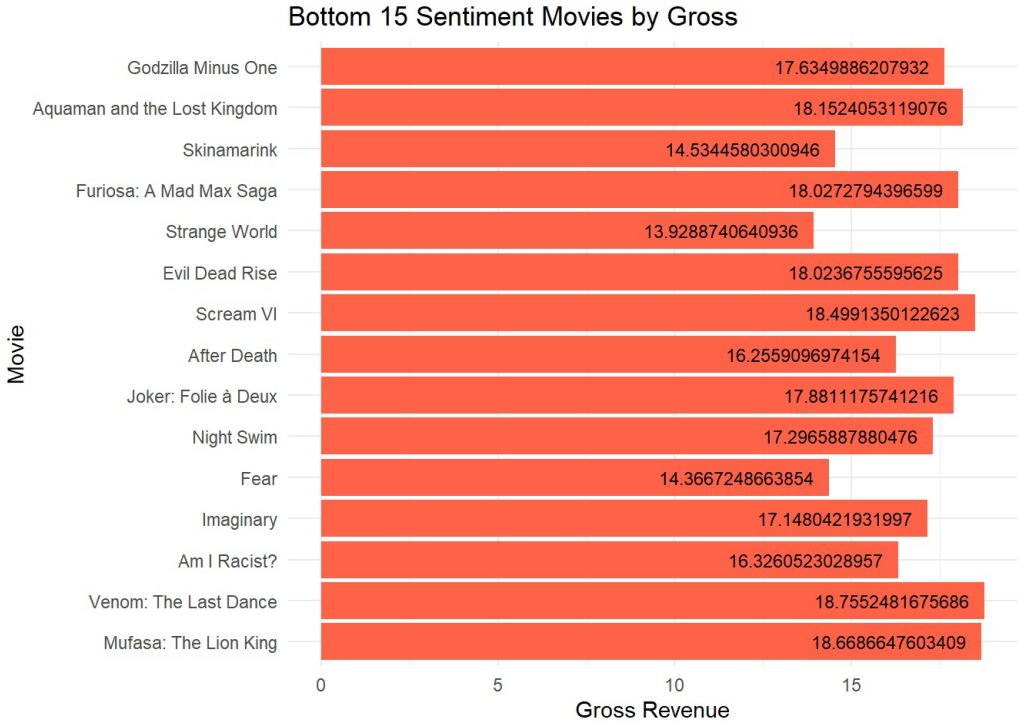

We can take a deeper look at the extremities of our data through top 15 histograms.

With the top sentiment score movies, the majority have low gross revenue values. Using the log() of the gross to normalize the gross revenue data. we can see that 8 of the movies do not break a log(gross) value of 17. In the bottom sentiment score movies, we can see that only 5 do not break the 17 mark. In our extreme samples, many of the lower sentiment movies actually perform better in the box office.

**Top 15 Sentiment Scored Movies by Gross Revenue:**

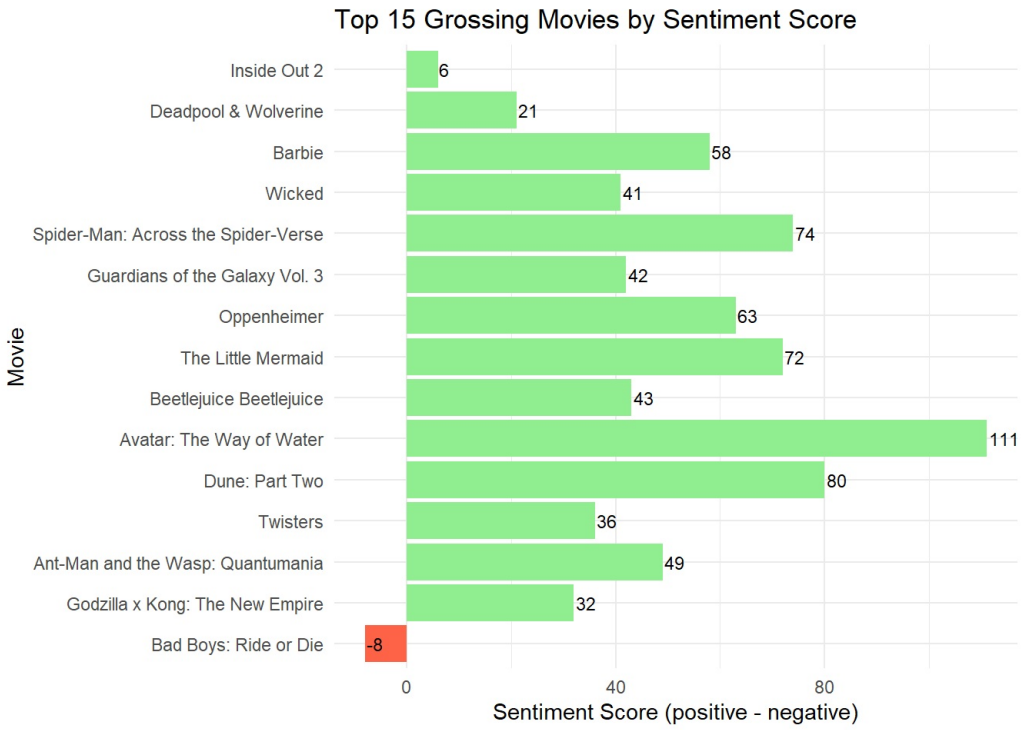



**Bottom 15 Sentiment Scored Movies by Gross Revenue:**

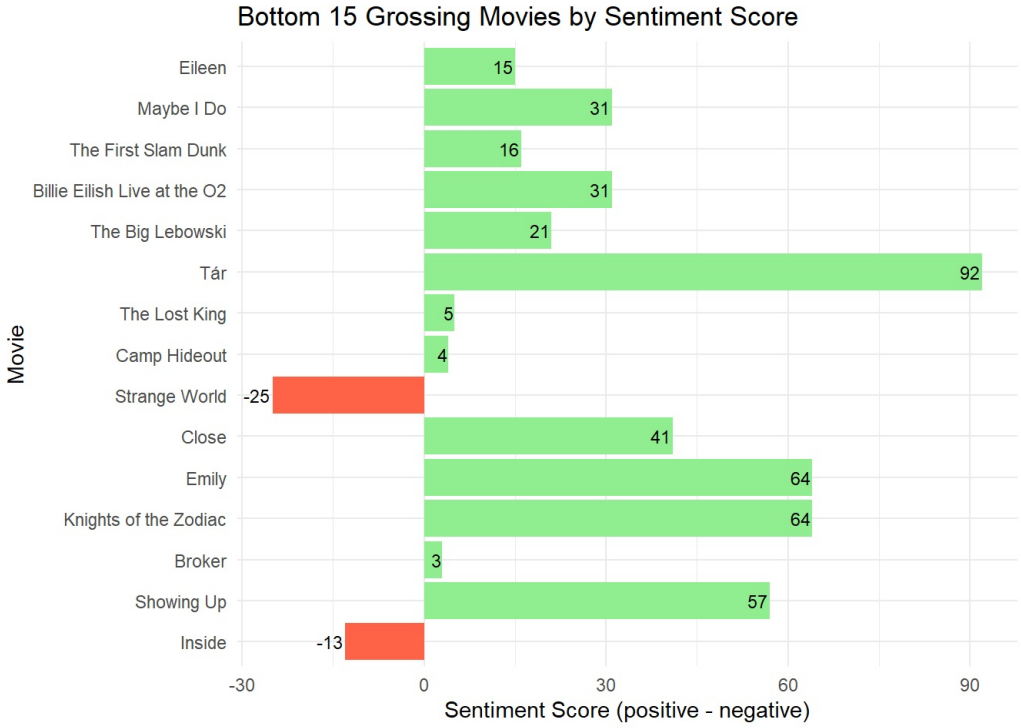



If we want to look at the top and bottom grossing movies by sentiment score, we can also see an interesting trend. The highest grossing movies, here have one value in the negative, and many values hovering around 40. The bottom grossing movies do have 2 movies in the negative, but the rest with a positive score, although many of the values being lower. This shows that many of the top grossing movies do recieve positive sentiment scores, although not the best.

**Top 15 Grossing Movies by Sentiment Score:**




**Bottom 15 Grossing Movies by Sentiment Score:**

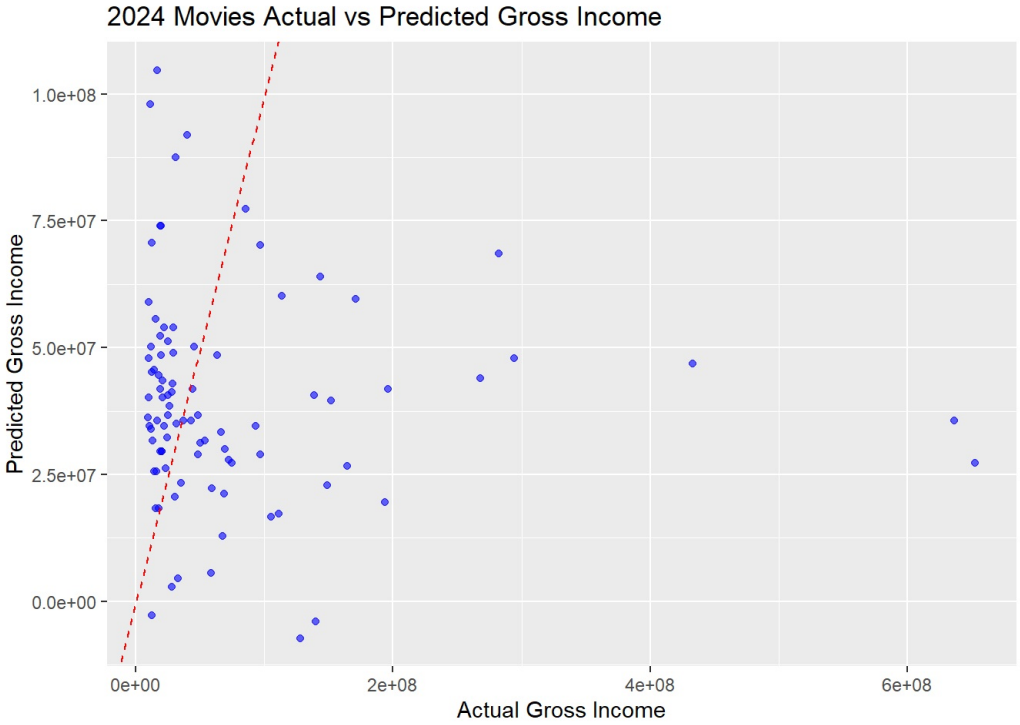



Finally we can take a look at our model that tries to predict the gross revenue using the sentiment scores. We trained this model on the 2023 data to predict the 2024 gross revenue using it's sentiment scores.

Looking at the summary of the model trained on our 2023 data, the p value for score is at 0.00316 which means at a 0.01 level it does have some impact on the gross revenue. For every 1 point of sentiment score increase, the revenue will increase by $557,091. However, looking at the residuals we have a minimum of -$90,268,315 and a maximum of $579,969,546, indicating that we have some extreme outliers. Also our R^2 value of 0.049 indicates that the model is not a good fit for the data, since a value of 1 would be ideal.

```
##
## Call:
## lm(formula = gross ~ score, data = movies_2023_tidy)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -90268315 -38678818 -21693378   3386243 579969546
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 23945186    8929906   2.681  0.00803 **
## score         557091     186109   2.993  0.00316 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 79410000 on 175 degrees of freedom
## Multiple R-squared:  0.04871,    Adjusted R-squared:  0.04327
## F-statistic:  8.96 on 1 and 175 DF,  p-value: 0.003159
```

We can use this model to predict 2024 movie's gross revenue by it's YouTube trailer sentiment scores. Looking at the plot below, we can see that the points do not fall on the line projected from our linear model. Showing that this model is not a good fit for our data and cannot predict gross income.




Please refer to Appendix C for more reproducible code relating to the EDA.

# Key Insights & Discussion

From our EDA, we can learn that using the sentiment scores of movie trailers from YouTube to figure out if a movie will be successful is not straightforward. Being a movie with a high sentiment value does not necessarily mean that you have a high grossing movie. And having a lower sentiment value does not necessarily mean you will have a low grossing movie. We also learned that the trend of sentiment and gross revenue can vary over different years. Thus, using one year of sentiment and gross data to predict the next year's may not be a viable strategy.

In order to practically use our findings, one may need to create a more complex model that includes other covariates such as ticket price, genre of movie, time of year, and how many advertisements were run before the movie release. Using this analysis in conjuction with the more complex model could be more useful in predicting whether a movie would be successful or not before release. This could be true in producing a movie or a product, where there are most likely mose covariates that are in play.

The main difficulties that were encountered during data cleaning mostly came from the YouTube comments. The YouTube API did not give us very many options in what comments we could download, so we had to pick the most relevant, according to the API, and then filter on date. We could not pick a date and get comments explicitly from then. We also could not find a dataset for YouTube links, so we needed to complete that task manually. Another issue that arose when planning the project and our analysis, was utilizing budget values to see whether or not a movie made or lost money. This information would have been better at determining whether or not a movie was successful or not. Looking at soley gross revenue is not ideal, since it could have cost the studio much more to make the film.

Again, for future analysis we might be able to expand the model to include more covariates and see if we can predict a movie's gross revenue better. We could also try to find a subset of movies that have reported their budget values to see whether they made or lost money. We might also be able to expand our dataset more and find more revenue values for more years to create a better trend.

# Conclusion

Recap the project workflow (data wrangling, merging, and analysis). Emphasize the most significant insights gained. Provide suggestions for further work or recommendations based on findings.

Our project set out to answer the question: How can we predict the overall gross revenue of a movie before it is released by using sentiment analysis from pre-release trailers? We utilized the Box Office Mojo gross revenue data for movies from 2023 and 2024, and combined it with YouTube comment data from each movie's trailer. Using both of these years, we could see that higher sentiment movies do not always imply higher grossing, as well as lower sentiment does not imply lower grossing. Many movies that had very high sentiment values, did not top the charts in the box office revenue. Our model trained on 2023 data was also unsuccessful in predicting 2024 gross revenue on it's sentiment scores. In future work, we could expand how we try to predict the gross revenue of movies by incorporating other covariates such as genre and number of advertisements that may have a more significant impact.

# References

Cite all data sources, articles, and tools used in the project.

[1] https://en.wikipedia.org/wiki/List_of_biggest_box-office_bombs (https://en.wikipedia.org/wiki/List_of_biggest_box-office_bombs)

[2] https://www.cnn.com/2019/04/30/entertainment/sonic-the-hedgehog-trailer-reactions-trnd/index.html (https://www.cnn.com/2019/04/30/entertainment/sonic-the-hedgehog-trailer-reactions-trnd/index.html)

[3] https://deadline.com/2025/01/sonic-the-hedgehog-franchise-one-billion-global-box-office-milestone-1236246684/ (https://deadline.com/2025/01/sonic-the-hedgehog-franchise-one-billion-global-box-office-milestone-1236246684/)

[4] https://console.cloud.google.com/marketplace/product/google/youtube.googleapis.com?inv=1&invt=Abww2A&authuser=2&project=dw-proj-457014 (https://console.cloud.google.com/marketplace/product/google/youtube.googleapis.com?inv=1&invt=Abww2A&authuser=2&project=dw-proj-457014)

[5] https://www.youtube.com/ (https://www.youtube.com/)

[6] https://www.boxofficemojo.com/year/2023/ (https://www.boxofficemojo.com/year/2023/)

[7] https://www.boxofficemojo.com/year/2024/ (https://www.boxofficemojo.com/year/2024/)

[8] https://www.kaggle.com/datasets/stefanoleone992/filmtv-movies-dataset (https://www.kaggle.com/datasets/stefanoleone992/filmtv-movies-dataset)

# Appendix A: Data Sources & Schema

**1. Data Sources**

- Box Office Mojo 2023
- Box Office Mojo 2024
- YouTube API
- Manual YouTube Codes 2023
- Manual YouTube Codes 2024
- FilmTV Movies Dataset from Kaggle

**2. Data Schema:**

- movie (string): movie title
- release date (date): release date of movie
- gross (int): gross revenue of movie
- code (string): code of YouTube video
- data (list): either comment data or word data once de-nested
- score (int): difference in bing sentiment score

# Appendix B: Data Cleaning & Transformation Steps

**2023 Data Cleaning & Transformation**

*1. Standardizing Box Office Mojo*

```
url="https://www.boxofficemojo.com/year/2023/"
movies_2023 <- url %>% read_html() %>% html_elements("table")%>% html_table(fill = TRUE) %>% pluck(1)
movies_2023_MGRD <- movies_2023 %>% mutate(release_date = as.Date(paste(movies_2023$`Release Date`, "2023"),"%b %
d %Y")) %>%
  mutate(gross = as.numeric(str_remove_all(Gross,"[$,]"))) %>% select(movie = "Release",gross,release_date)
```

We need to make sure that the data string is in the date format of month, day, and year, remove spaces and other characters from it, make sure it is all numbers, and then keep the post date so that we can filter the YouTube comments based on that date.

*2. Manual YouTube Links and Joining Mojo Data*

```
#read the sheet from kaggle
movies_2023_MGRDL <- read_csv("C:/Users/polynLin/Desktop/mypart/movies&link&gross&releaseddate.csv")
#combine these two sheet into one
colnames(movies_2023_MGRDL) <- tolower(colnames(movies_2023_MGRDL))
colnames(movies_2023_MGRD) <- tolower(colnames(movies_2023_MGRD))

movies_2023_MGRDL <- movies_2023_MGRDL %>%
  mutate(release_date = as.Date(release_date, format = "%d/%m/%Y"))
movies_2023_MGRD <- movies_2023_MGRD %>%
  mutate(release_date = as.Date(release_date))
# combine them and save the value that are similar, and keep the values from scraped sheet if any two values are
not the same
#the new sheet has movie, gross, release_date, link
movies_merged_final<- full_join(movies_2023_MGRDL, movies_2023_MGRD, by = "movie") %>%
  mutate(
    gross = coalesce(movies_2023_MGRD$gross, movies_2023_MGRDL$gross),
    release_date = coalesce(movies_2023_MGRD$release_date, movies_2023_MGRDL$release_date),
    link = movies_2023_MGRDL$link
  ) %>%
  select(movie, gross, release_date, link)
```

I first found the 2023 movie data table from Kaggle, then manually searched for the movie's corresponding YouTube trailer video ID (i.e. link) based on the movie name, then compared and merged the 2023 movie data captured from Mojo with the former, and finally filtered out the movie name, gross, date and link.

*3. YouTube Comments*

```
movies_merged_final$release_date <- as.Date(movies_merged_final$release_date)
movies_merged_final$data <- vector("list", nrow(movies_merged_final))
api_key = "AIzaSyDIjgqQFmPSa9ExEfy0mHDYI7KOfO-M3OA"
# clean the link values
movies_merged_final$link <- gsub('^["\']+|["\']+$', '', movies_merged_final$link)
movies_merged_final <- movies_merged_final %>%
  filter(tolower(link) != "link")
#scarping comments
for (i in 1:nrow(movies_merged_final)) {
  cat("Processing:", movies_merged_final$movie[i], "\n")
  video_id <- movies_merged_final$link[i]
  release_date <- movies_merged_final$release_date[i]

  url <- paste0("https://www.googleapis.com/youtube/v3/commentThreads?part=snippet&videoId=",
                video_id, "&maxResults=100&order=relevance&key=", api_key)
  res <- tryCatch(GET(url), error = function(e) return(NULL))
  if (!is.null(res) && status_code(res) == 200) {
    data <- fromJSON(content(res, "text", encoding = "UTF-8"))

    if (!is.null(data$items) && length(data$items) > 0) {
      comments <- data.frame(
        DatePosted = as.Date(sapply(data$items$snippet$topLevelComment$snippet$publishedAt, substr, 1, 10)),
        Comment = sapply(data$items$snippet$topLevelComment$snippet$textDisplay, identity),
        stringsAsFactors = FALSE
      )
      date_filtered <- comments %>%
        filter(DatePosted < release_date)
      #filter the movies
      movies_merged_final$data[[i]] <- date_filtered
    }
  }
}
movies_merged_final$comment_count <- sapply(movies_merged_final$data, function(x) length(x$Comment))

# filter comments==0 or movies include "re-release"
filtered_movies <- movies_merged_final %>%
  filter(comment_count > 0) %>%
  filter(!grepl("re[- ]release", movie, ignore.case = TRUE)) %>%
  arrange(comment_count) %>%
  select(movie, comment_count)
head(filtered_movies, 10)
```

I used the movie trailer ID to fetch comments on YouTube for the day the movie trailer was released through the API. I decided to filter out some movies because they were re-released or the trailer comments section was closed.

*4. De-Nesting Words from Comments*

```
bing <- get_sentiments("bing")
data("stop_words")
all_comments <- movies_merged_final %>%
  filter(lengths(data) > 0) %>%
  select(movie, data) %>%
  unnest(cols = c(data)) %>%
  filter(!is.na(Comment) & Comment != "")
comments_tokenized <- all_comments %>%
  unnest_tokens(word, Comment) %>%
  anti_join(stop_words, by = "word") %>%
  inner_join(bing, by = "word")
```

Load the sentiment dictionary and stop word list and then extract all non-empty comment texts, tokenize the comments, and remove meaningless stop words. Filter out words with sentiment tendencies. The final result is a data frame containing all sentiment words and their corresponding sentiment labels that appear in the comments.

**2024 Data Cleaning & Transformation**

*1. Standardizing Box Office Mojo*

```
movies_mojo_2024_tidy <- movies_mojo_2024 %>% mutate(release_date = as.Date(paste(movies_mojo_2024$`Release Date`
, "2024"),"%b %d %Y")) %>%
  mutate(gross = as.numeric(str_remove_all(Gross,"[$,]"))) %>% select(movie = "Release",gross,release_date)
```

Need to make sure the data string is in a date format, specifically month, day, year.

Remove the $ and , from the gross value to make it a number.

Keep the release date as well so we can filter on it for YouTube comments.

*2. Manual YouTube Links*

```
youtube_links <- readxl::read_xlsx("data/Youtube Links 2024.xlsx") %>%
  mutate(code = str_remove_all(link, "https?://(www\\.)?youtube\\.com/watch\\?v=")) %>%
  select(movie,code)
```

Make sure to remove the front part of the URL to get the code for the API.

*3. YouTube Comments*

```
for (i in 1:nrow(movies_mojo_codes)) {
  url <- paste0("https://www.googleapis.com/youtube/v3/commentThreads?part=snippet&videoId=",
  movies_mojo_codes$code[i], "&maxResults=100&order=relevance&key=", api_key)

  res <- GET(url)
  data <- fromJSON(content(res, "text"))

  comments <- data.frame(
    DatePosted = as.Date(unlist(data$items$snippet$topLevelComment$snippet$publishedAt)),
    Comment = unlist(data$items$snippet$topLevelComment$snippet$textDisplay)
  )

  date_filtered <- comments %>%
    filter(DatePosted < movies_mojo_codes$release_date[i])

  movies_gross_comments$data[i] <- list(date_filtered)
}
```

From the YouTube API data, we must extract the comments labeled “topLevelComment$snippet$textDisplay” and the comment published date of “topLevelComment$snippet$publishedAt”.

We also need to filter the comments on the release date of the movie coming from the Box Office Mojo data.

*4. De-Nesting Words from Comments*

```
for (i in 1:nrow(movies_gross_comments)) {
  movies_sentiment$data[[i]] <- movies_gross_comments$data[[i]] %>% select(Comment) %>%
    mutate(Comment = str_remove_all(Comment,"<a[^>]*?>.*?</a>")) %>%
    unnest_tokens(word, Comment) %>%
    anti_join(stop_words) %>%
    mutate(word = str_replace_all(word,"[^[:alpha:]\\s]","")) %>%
    filter(word != "")
}
```

We need to de-nest the words from the comments gathered from the YouTube API.

We want to remove all timestamp values, denoted with …, stop words, as well as remove all non-words such as emojis or other characters.

# Appendix C: Code for Reproducability

GitHub Repository: https://github.com/thomas-stranick-RU/DW-Project (https://github.com/thomas-stranick-RU/DW-Project)

**2023 Tidying Code for Reproducability**

```
library(tidyverse)
library(httr)
library(jsonlite)
library(tidytext)
library(usethis)
library(dplyr)
library(readr)
library(rvest)

#1
# scraping data from mojo and save as a file
url="https://www.boxofficemojo.com/year/2023/"

movies_2023 <- url %>% read_html() %>% html_elements("table")%>% html_table(fill = TRUE) %>% pluck(1)

#movies_2023
# filter movie, gross, release_date and save as a new file
movies_2023_MGRD <- movies_2023 %>% mutate(release_date = as.Date(paste(movies_2023$`Release Date`, "2023"),"%b %
d %Y")) %>%
  mutate(gross = as.numeric(str_remove_all(Gross,"[$,]"))) %>% select(movie = "Release",gross,release_date)

#movies_2023_MGRD
#write_csv(movies_2023_MGRD, "C:/Users/polynLin/Desktop/mypart/movies_2023_MGRD.csv")

#read the sheet
movies_2023_MGRDL <- read_csv("data/movies&link&gross&releaseddate.csv")

#combine these two sheet into one
colnames(movies_2023_MGRDL) <- tolower(colnames(movies_2023_MGRDL))
colnames(movies_2023_MGRD) <- tolower(colnames(movies_2023_MGRD))

movies_2023_MGRDL <- movies_2023_MGRDL %>%
  mutate(release_date = as.Date(release_date, format = "%d/%m/%Y"))
movies_2023_MGRD <- movies_2023_MGRD %>%
  mutate(release_date = as.Date(release_date))
# combine them and save the value that are similar, and keep the values from scraped sheet if any two values are
not the same
#the new sheet has movie, gross, release_date, link
movies_merged_final<- full_join(movies_2023_MGRDL, movies_2023_MGRD, by = "movie") %>%
  mutate(
    gross = coalesce(movies_2023_MGRD$gross, movies_2023_MGRDL$gross),
    release_date = coalesce(movies_2023_MGRD$release_date, movies_2023_MGRDL$release_date),
    link = movies_2023_MGRDL$link
  ) %>%
  select(movie, gross, release_date, link)

# save as a new CSV
#write_csv(movies_merged_final, "C:/Users/polynLin/Desktop/mypart/movies_merged_final.csv")


#2scraping comments from YouTube
#getting the data from API
library(httr)
library(jsonlite)
library(dplyr)

movies_merged_final$release_date <- as.Date(movies_merged_final$release_date)

movies_merged_final$data <- vector("list", nrow(movies_merged_final))

api_key = "AIzaSyDIjgqQFmPSa9ExEfy0mHDYI7KOfO-M3OA"

# clean the link values
movies_merged_final$link <- gsub('^["\']+|["\']+$', '', movies_merged_final$link)

movies_merged_final <- movies_merged_final %>%
  filter(tolower(link) != "link")

#scarping comments
for (i in 1:nrow(movies_merged_final)) {
  cat("Processing:", movies_merged_final$movie[i], "\n")
```

```
  video_id <- movies_merged_final$link[i]
  release_date <- movies_merged_final$release_date[i]

  url <- paste0("https://www.googleapis.com/youtube/v3/commentThreads?part=snippet&videoId=",
                video_id, "&maxResults=100&order=relevance&key=", api_key)

  res <- tryCatch(GET(url), error = function(e) return(NULL))

  if (!is.null(res) && status_code(res) == 200) {
    data <- fromJSON(content(res, "text", encoding = "UTF-8"))

    if (!is.null(data$items) && length(data$items) > 0) {
      comments <- data.frame(
        DatePosted = as.Date(sapply(data$items$snippet$topLevelComment$snippet$publishedAt, substr, 1, 10)),
        Comment = sapply(data$items$snippet$topLevelComment$snippet$textDisplay, identity),
        stringsAsFactors = FALSE
      )

      date_filtered <- comments %>%
        filter(DatePosted < release_date)
      #filter the movies
      movies_merged_final$data[[i]] <- date_filtered
    }
  }
}

movies_merged_final

# check the comments number of movies
#sorted_movies <- movies_merged_final %>%
#  arrange(comment_count) %>%
#  select(movie, comment_count)
#head(sorted_movies, 10)

# renew the number of comments
movies_merged_final$comment_count <- sapply(movies_merged_final$data, function(x) length(x$Comment))

# filter comments==0 or movies include "re-release"
filtered_movies <- movies_merged_final %>%
  filter(comment_count > 0) %>%
  filter(!grepl("re[- ]release", movie, ignore.case = TRUE)) %>%
  arrange(comment_count) %>%
  select(movie, comment_count)
head(filtered_movies, 10)
#write_csv(filtered_movies, "C:/Users/polynLin/Desktop/mypart/filtered_movies.csv")

#3
library(tidytext)
library(dplyr)
library(tidyr)

# loading "bing" stop words
bing <- get_sentiments("bing")
data("stop_words")
# combine all these comments
all_comments <- movies_merged_final %>%
  filter(lengths(data) > 0) %>%
  select(movie, data) %>%
  unnest(cols = c(data)) %>%
  filter(!is.na(Comment) & Comment != "")
# split comments
comments_tokenized <- all_comments %>%
  unnest_tokens(word, Comment) %>%
  anti_join(stop_words, by = "word") %>%
  inner_join(bing, by = "word")
# count the numbers of words
movie_sentiment_stats <- comments_tokenized %>%
  count(movie, sentiment) %>%
  pivot_wider(names_from = sentiment, values_from = n, values_fill = 0) %>%
  mutate(
    sentiment_score = positive - negative,
    total_words = positive + negative,
    avg_sentiment = sentiment_score / total_words
  ) %>%
  arrange(desc(avg_sentiment))

movies_final_sentiment <- filtered_movies %>%
  left_join(movie_sentiment_stats, by = "movie")
head(movies_final_sentiment, 10)
```

```
movies_final_model <- movies_final_sentiment %>%
  left_join(movies_merged_final %>% select(movie, gross), by = "movie") %>%
  filter(!is.na(sentiment_score), !is.na(gross), gross > 0) %>%
  mutate(log_gross = log(gross))

movies_final_model

#write_csv(movies_final_model, "data/movies_2023_tidy")
```

**2024 Tidying Code for Reproducability**

```
library(tidyverse)
library(httr)
library(jsonlite)
library(tidytext)
library(rvest)

#2024 Cleaning

#Box Office Mojo: Box Office Numbers

url="https://www.boxofficemojo.com/year/2024/"

#Web scraping site for top grossing movies list
movies_mojo_2024 <- url %>% read_html() %>% html_elements("table")%>% html_table(fill = TRUE) %>% pluck(1)

#Data tidying; updating release date, gross revenue
movies_mojo_2024_tidy <- movies_mojo_2024 %>% mutate(release_date = as.Date(paste(movies_mojo_2024$`Release Date`
, "2024"),"%b %d %Y")) %>%
  mutate(gross = as.numeric(str_remove_all(Gross,"[$,]"))) %>% select(movie = "Release",gross,release_date)

#YouTube Codes
youtube_links <- readxl::read_xlsx("data/Youtube Links 2024.xlsx") %>%
  mutate(code = str_remove_all(link, "https?://(www\\.)?youtube\\.com/watch\\?v=")) %>%
  select(movie,code)

#left join to merge mojo table with YouTube codes
movies_mojo_codes <- left_join(youtube_links,movies_mojo_2024_tidy, by= join_by(movie))

#YouTube API
api_key = "AIzaSyDIjgqQFmPSa9ExEfy0mHDYI7KOfO-M3OA"

movies_gross_comments <- movies_mojo_codes %>% select(movie,gross)

#for each movie, getting comments, filtering by release date of movie
for (i in 1:nrow(movies_mojo_codes)) {
  url <- paste0("https://www.googleapis.com/youtube/v3/commentThreads?part=snippet&videoId=",
                movies_mojo_codes$code[i], "&maxResults=100&order=relevance&key=", api_key)

  res <- GET(url)
  data <- fromJSON(content(res, "text"))

  comments <- data.frame(
    DatePosted = as.Date(unlist(data$items$snippet$topLevelComment$snippet$publishedAt)),
    Comment = unlist(data$items$snippet$topLevelComment$snippet$textDisplay)
  )

  date_filtered <- comments %>%
    filter(DatePosted < movies_mojo_codes$release_date[i])

  movies_gross_comments$data[i] <- list(date_filtered)
}

#head(movies_gross_comments$data[[1]],3)

#removing release date and YouTube code
movies_sentiment <- movies_gross_comments %>% select(movie,gross)

#denest by token and remove all timestamps, non-words, and stop words
for (i in 1:nrow(movies_gross_comments)) {
  movies_sentiment$data[[i]] <- movies_gross_comments$data[[i]] %>% select(Comment) %>%
    mutate(Comment = str_remove_all(Comment,"<a[^>]*?>.*?</a>")) %>%
    unnest_tokens(word, Comment) %>%
    anti_join(stop_words) %>%
    mutate(word = str_replace_all(word,"[^[:alpha:]\\s]","")) %>%
    filter(word != "")
}
```

```
#get the overall sentiment score with bing
bing_sentiment <- get_sentiments("bing")
for (i in 1:nrow(movies_sentiment)) {
  sentiment_scores <- movies_sentiment$data[[i]] %>%
    inner_join(bing_sentiment, by = "word") %>%
    count(sentiment)
  movies_sentiment$score[i] <- sentiment_scores$n[2] - sentiment_scores$n[1]
}
movies_2024 <- movies_sentiment %>% select(movie,gross,score)

write.csv(movies_2024,"data/movies_2024_tidy.csv")
```

**Sentiment Analysis and Model**

```
library(tidyverse)
library(ggplot2)
library(plotly)
library(ggrepel)

#Sentiment Analysis and Model

movies_2023_tidy <- tibble(read.csv("movies_2023_tidy") %>% select(movie,gross,score = sentiment_score))
movies_2023_tidy

movies_2024_tidy <- tibble(read.csv("movies_2024_tidy") %>% select(movie,gross,score))
movies_2024_tidy

movies_2023_combine <- movies_2023_tidy %>% mutate(year = 2023)
movies_2024_combine <- movies_2024_tidy %>% mutate(year = 2024)
combined_movies <- rbind(movies_2023_combine,movies_2024_combine)
combined_movies

#Trend of 2023 Data
trend_2023 <- ggplot(movies_2023_tidy, aes(x = score, y = log(gross))) +
  geom_point(color = "blue", alpha = 0.7) +
  geom_smooth(method = "lm", se = FALSE, color = "red") +
  labs(
    title = "2023 Sentiment Score vs Log(Gross)",
    x = "Sentiment Score",
    y = "Log(Gross Revenue)"
  ) +
  theme_minimal()

#2024 trend
ggplot(movies_2024_tidy, aes(x = score, y = log(gross))) +
  geom_point(color = "blue", alpha = 0.7) +
  geom_smooth(method = "lm", se = FALSE, color = "red") +
  labs(
    title = "2024 Sentiment Score vs Log(Gross)",
    x = "Sentiment Score",
    y = "Log(Gross Revenue)"
  ) +
  theme_minimal()

#combined trend
extreme_points <- combined_movies %>%
  filter(score > quantile(score, 0.995) | score < quantile(score, 0.005) |
           gross > quantile(gross, 0.995) | gross < quantile(gross, 0.005))

 ggplot(combined_movies, aes(x = score, y = log(gross))) +
  geom_point(color = "blue", alpha = 0.7) +
  geom_smooth(method = "lm", se = FALSE, color = "red") +
   geom_point(data = extreme_points, aes(x = score, y = log(gross)),
              color = "orange", size = 3, shape = 17) +  # Highlight extreme points
   geom_text_repel(data = extreme_points, aes(x = score, y = log(gross), label = paste("\nMovie:", movie,"\nGross
:", gross, "\nScore:", score)),
             color = "black", size = 3, vjust = -0.5, hjust = 0.5) +
  labs(
    title = "2023-2024 Sentiment Score vs Log(Gross)",
    x = "Sentiment Score",
    y = "Log(Gross Revenue)"
  ) +
  theme_minimal()

#Select the top 15 sentiment by gross
top_sentiment_movies <- combined_movies %>%
  filter(!is.na(score)) %>%
```

```
  slice_max(order_by = score, n = 15)

top_sentiment_movies <- top_sentiment_movies[-12,]

ggplot(top_sentiment_movies, aes(x = reorder(movie, score), y = log(gross))) +
  geom_col(fill = "lightgreen") +
  geom_text(aes(label = log(gross)),
            hjust = -0.1,
            size = 3) +
  coord_flip() +
  labs(
    title = "Top 15 Sentiment Movies by Gross",
    x = "Movie",
    y = "Gross Revenue"
  ) +
  theme_minimal()

#The 15 lowest sentiment score
low_sentiment_movies <- combined_movies %>%
  filter(!is.na(score)) %>%
  slice_min(order_by = score, n = 15)

ggplot(low_sentiment_movies, aes(x = reorder(movie, score), y = log(gross))) +
  geom_col(fill = "tomato") +
  geom_text(aes(label = log(gross)),
            hjust = -0.1,
            size = 3) +
  coord_flip() +
  labs(
    title = "Bottom 15 Sentiment Movies by Gross ",
    x = "Movie",
    y = "Gross Revenue"
  ) +
  theme_minimal()


#Select the top 15 grossing by sentiment
top_sentiment_movies <- combined_movies %>%
  filter(!is.na(score)) %>%
  slice_max(order_by = gross, n = 15)

ggplot(top_sentiment_movies, aes(x = reorder(movie, gross), y = score)) +
  geom_col(aes(fill = score > 0)) +
  geom_text(aes(label = score),
            hjust = -0.1,
            size = 3) +
  scale_fill_manual(values = c("TRUE" = "lightgreen", "FALSE" = "tomato"), guide = "none") +
  coord_flip() +
  labs(
    title = "Top 15 Grossing Movies by Sentiment Score",
    x = "Movie",
    y = "Sentiment Score (positive - negative)"
  ) +
  theme_minimal()

#The 15 lowest grossing by sentiment
low_sentiment_movies <- combined_movies %>%
  filter(!is.na(score)) %>%
  slice_min(order_by = gross, n = 15)

ggplot(low_sentiment_movies, aes(x = reorder(movie, gross), y = score)) +
  geom_col(aes(fill = score > 0)) +
  geom_text(aes(label = score),
            hjust = 1.1,
            size = 3) +
  scale_fill_manual(values = c("TRUE" = "lightgreen", "FALSE" = "tomato"), guide = "none") +
  coord_flip() +
  labs(
    title = "Bottom 15 Grossing Movies by Sentiment Score",
    x = "Movie",
    y = "Sentiment Score (positive - negative)"
  ) +
  theme_minimal()

#Predictive linear model

model <- lm(gross ~ score, data = movies_2023_tidy)
summary(model)
```

```
prediction <- predict(model, newdata = movies_2024_tidy)

predicted <- data.frame(actual = movies_2024_tidy$gross, predicted = prediction)

ggplot(predicted,aes(x = actual, y = predicted)) +
  geom_point(color = "blue", alpha = 0.6) +
  geom_abline(intercept = 0, slope = 1, color = "red", linetype = "dashed") +
  labs(
    title = "2024 Movies Actual vs Predicted Gross Income",
    x = "Actual Gross Income",
    y = "Predicted Gross Income"
  )
```

# Appendix D: Data Analysis and Processing

**Data Cleaning**

*Movies with 0 Comments (comments turned off on YouTube Video)*

- 2023 - 33/200
- 2024 - 10/100

*Inconsistent Formats*

- 2023 - 0/200
- 2024 - 0/100

**Data Merging**

- Records before merge: 200 (2023), 100 (2024)
- Records after merge: 167 (2023), 90 (2024)
- Key matching columns: movie, gross, score, year

**EDA Adjustments**

- Normalization: Gross revenue normalized in some plots using log()
- Sentiment Calculation: calulated bing sentiment score for each movie